

Module 10: Interview Capstone

Same story throughout: student **Aisha** (roll 101) takes **DBMS 101**, taught by **Prof. Rao**, and gets grade **A**.

This module teaches **no new concepts**. It connects the seven you already have, so you can use them under pressure instead of reciting them.

10.1 How It All Connects — Core • 9 min

Hook

An interviewer asks one question: *"Walk me through how you'd design a database."*

You know ER models. You know normalization. You know indexes. But right now they're seven separate boxes in your head, and the question needs one line of thinking.

Why + What

Here's the thing nobody tells you: the modules aren't a list. **Each one exists because the previous one caused a problem.**

Once you see that chain, you can start anywhere and walk in either direction — which is exactly what an interview requires.

The whole course is one pipeline: model the world, turn it into tables, split the tables, query them, protect them from each other, then make them fast.

How It Works (with the easiest example)

Follow one fact — *Aisha got an A in DBMS 101* — through every module.

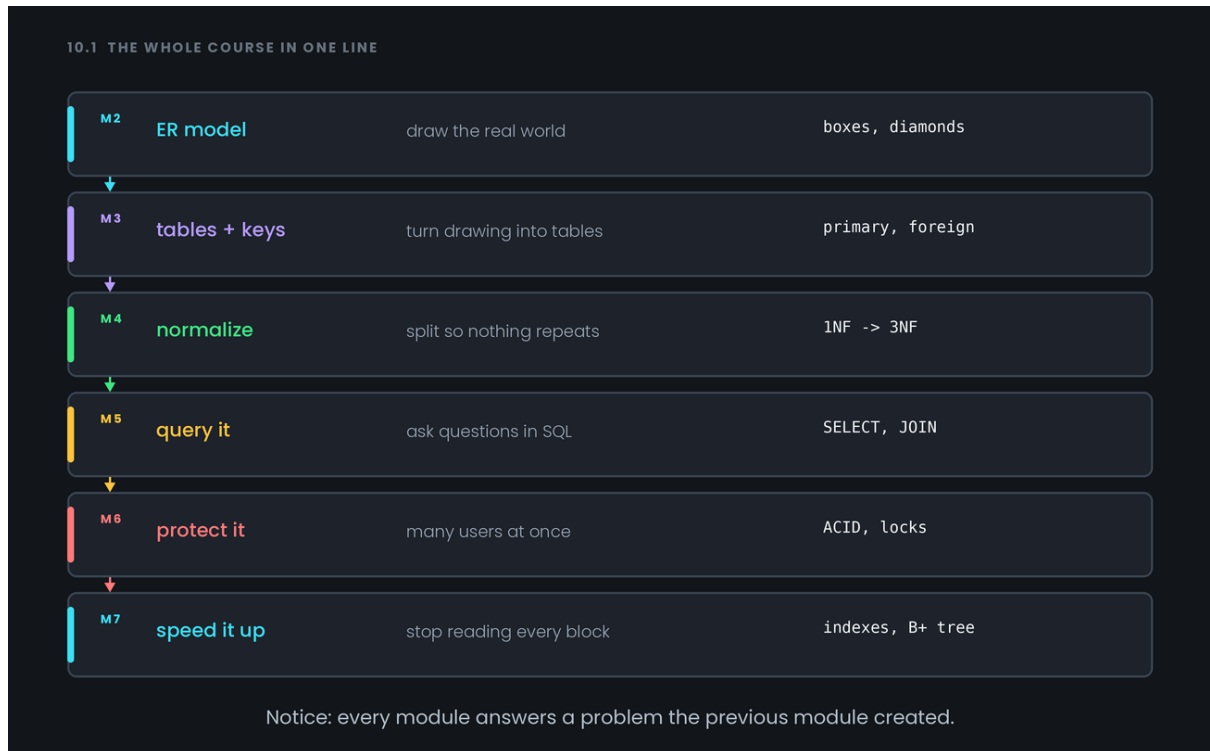
1. **Module 2 gave you the picture.** Student and Course are entities, "enrolls in" is a relationship, and it's many-to-many. **The problem it created:** a picture can't store anything.
2. **Module 3 turned it into tables.** Student, Course, Enrollment — with roll_no as a primary key and foreign keys linking them, plus constraints refusing roll 999. **The problem it created:** nothing yet stops you putting everything in one wide table.
3. **Module 4 split them properly.** Prof. Rao's name lives in one place, so renaming him is one update. **The problem it created:** the data you want is now scattered across four tables.
4. **Module 5 put it back together on demand.** Joins reassemble the scattered pieces; GROUP BY answers "how many per course." **The problem it created:** 900 students are running these queries simultaneously.
5. **Modules 6 and 7 handled the crowd and the clock.** Transactions and locks stop concurrent users corrupting each other. Indexes stop every query reading every block.

Read backwards and it works too: *"Why do we need an index?"* → because tables got big → because we split them properly → because one big table caused anomalies.

Pause here. In one sentence each, why does normalization make joins necessary, and why do joins make indexes matter?

Normalization splits one table into several, so the data you want lives in more than one place — joins put it back. And joins match rows across tables, so without an index each match means scanning every block.

Visualize



Notice: every module answers a problem the previous module created.

Common Confusion

Students treat these as unrelated exam topics and get stuck the moment a question crosses a boundary — like *"how does normalization affect performance?"*

That question has an easy answer once you see the chain: normalization removes repeated data, which means more tables, which means more joins, which means more reads — so you add indexes to pay that cost back, or denormalize deliberately if you've measured that you must.

That answer touches Modules 4, 5 and 7 in one breath. That's what a strong candidate sounds like.

Interview Angle

1. *"How does normalization affect query performance?"* — More tables, more joins, slower reads — repaid with indexes, or with deliberate denormalization when measurement justifies it.

2. *"Why does a primary key make lookups fast?"* — Because databases build an index on it automatically, so it's a B+ tree lookup rather than a full table scan. This connects Modules 3 and 7.
3. *"What's the relationship between an ER diagram and the tables you finally build?"* — Entities become tables, attributes become columns, M:N relationships become their own junction table.

Recap: The course is one chain — model, tabulate, normalize, query, protect, optimise — where each module solves the problem the one before it created.

10.2 The 5-Minute Design Answer — Core • 10 min

Hook

"Design a database for a library."

You have five minutes and a whiteboard. Most candidates start drawing tables immediately and forget half of what they know.

There's a better opening move.

Why + What

Design questions aren't testing whether you can invent a schema. They're testing whether you have a **method** — and whether you mention the things beginners forget: cardinality, normal form, transactions, indexes.

A design answer has four checks after the tables: what are the cardinalities, is it normalized, what needs to be one transaction, and what needs an index.

Say those four out loud and you've covered Modules 2, 4, 6 and 7 without being asked.

How It Works (with the easiest example)

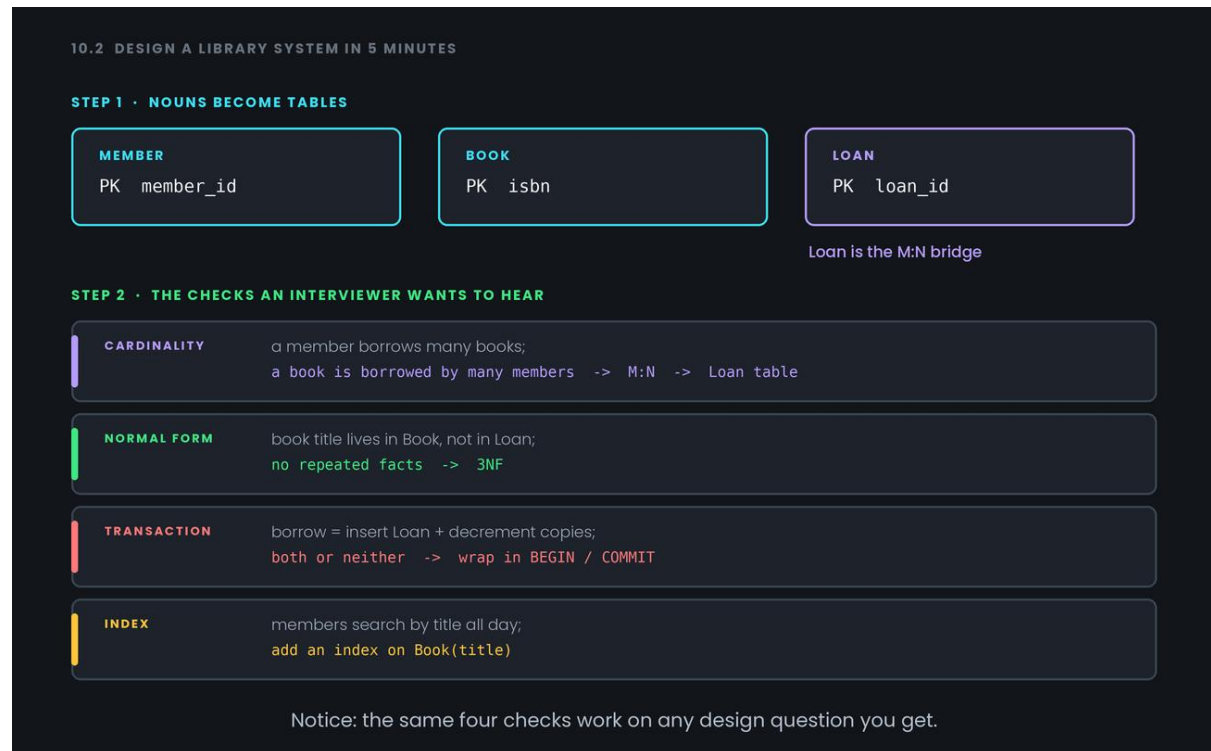
The library question, answered properly.

1. **Find the nouns → tables.** Member, Book. Each gets a primary key: member_id, isbn. Say the keys out loud; candidates who skip them get asked anyway.
2. **Find the verbs → relationships, then check cardinality.** A member borrows many books. A book is borrowed by many members over time. **Many-to-many** — which means a third table, Loan(loan_id, member_id, isbn, issue_date, due_date).
3. **Check normal form.** Is the book's title stored in Loan? It shouldn't be — it belongs in Book, because title depends on the book, not the loan. Say "so we're in 3NF" and move on. One sentence.
4. **Find what must be all-or-nothing.** Borrowing is two changes: insert a Loan row, and decrease the available copy count. Half of that is a broken library. **Wrap it in a transaction.**
5. **Find the hot query, then index it.** Members search by title constantly. Add an index on Book(title). Mention the cost — writes get slower — so it's clear you know it isn't free.

Pause here. The library adds fines. Where does fine_amount belong — Member, Book, or Loan?

Loan. A fine comes from one specific late return, so it depends on the loan — not on the member generally, and not on the book. Putting it in Member would break as soon as someone has two overdue books.

Visualize



Notice: the same four checks work on any design question you get.

Common Confusion

Candidates think a design answer means "produce the perfect schema." It doesn't. Interviewers are watching **how you think**, and they will deliberately change the requirements mid-answer to see if your design bends.

So narrate as you go. "I'm making this many-to-many because a book can be borrowed by different people over time — if that weren't true, I'd put a foreign key in Book instead." That one sentence shows you understand cardinality far better than a correct diagram drawn in silence.

Second: **don't skip the boring tables.** A junction table is the single most common thing candidates forget, and it's the thing that proves you understood M:N.

Interview Angle

1. "Design a database for [anything]." — Nouns to tables, verbs to relationships, cardinality on each one, then the four checks. Narrate your reasoning throughout.
2. "What would you change if the system had 10 million users?" — Indexes on the hot lookups, possibly a denormalized reporting table, and shorter transactions to reduce lock contention.

3. "Where would you put this new column?" — Answer with a dependency: it goes in the table whose key actually decides it. That's the 3NF reasoning from Module 4, applied live.

Recap: Answer any design question with nouns-to-tables, then four checks — cardinality, normal form, what must be one transaction, and what needs an index — narrating your reasoning as you go.

10.3 Rapid-Fire Revision — Core • 8 min

Hook

The last ten minutes of an interview are usually short, sharp questions. No whiteboard, no thinking time.

Almost all of them come from six **pairs** of concepts that sound similar and aren't.

Why + What

Interviewers use pairs because a candidate who memorised definitions can recite both — but can't say what **one word** separates them.

Every pair below differs by a single idea. Learn that idea, and both halves come free.

How It Works (with the easiest example)

Read each as *difference first*, then the pair.

1. **Super key vs candidate key** → the word is **minimal**. Both are unique; a candidate key has nothing spare. (roll_no, name) is a super key only.
2. **2NF vs 3NF** → the word is **what it depends on**. 2NF: depends on part of the key. 3NF: depends on a non-key column.
3. **DELETE vs TRUNCATE** → the word is **rollback**. DELETE picks rows and can be rolled back; TRUNCATE removes all rows and is DDL.
4. **WHERE vs HAVING** → the word is **when**. WHERE runs before grouping, HAVING after.
5. **Non-repeatable vs phantom read** → the word is **changed vs appeared**. A row you read changed, versus a new row showing up.
6. **Clustered vs non-clustered** → the word is **physical**. Clustered sets the actual row order, so one per table.

Pause here. Cover the right-hand side of the diagram and say the missing half of all six pairs out loud. If any one takes more than five seconds, that's the topic to reread tonight.

Visualize

10.3 THE PAIRS INTERVIEWERS USE TO CATCH YOU

SUPER KEY / CANDIDATE KEY unique	unique AND minimal
2NF / 3NF depends on part of the key	depends on a non-key column
DELETE / TRUNCATE rows, can roll back	all rows, DDL
WHERE / HAVING filters rows before grouping	filters groups after
NON-REPEATABLE / PHANTOM a row you read changed	a new row appeared
CLUSTERED / NON-CLUSTERED the rows ARE the order	a separate list that points back

Notice: each pair differs by exactly one word — learn the word, not the pair.

Notice: each pair differs by exactly one word — learn the word, not the pair.

Common Confusion

The trap here isn't a concept — it's **overconfidence in recognition**. Reading "ah yes, WHERE and HAVING" feels like knowing it. Saying the difference out loud, cold, in eight seconds, is a different skill.

Test yourself by producing the answer, not recognising it.

Interview Angle

Ten questions that cross module boundaries. If you can answer these, the course has done its job.

1. *"What's the difference between a primary key and a unique constraint?"* — Both enforce uniqueness; a primary key also forbids NULL, and there's only one per table.
2. *"Why is a foreign key allowed to be NULL?"* — It means "not linked to anything yet," which is a legitimate state. A primary key can never be NULL, because a row must be identifiable.
3. *"Your query got slow after the table grew. What do you check?"* — Whether it's doing a full scan, whether a useful index exists, and whether a function on the column is disabling one.
4. *"Is a table in 3NF always in BCNF?"* — No. Give the (student, subject, teacher) case where a non-key column decides another.

5. *"How would you find rows in A with no match in B?"* — LEFT JOIN, then WHERE b.column IS NULL.
6. *"What happens if two users update the same row at the same time?"* — Locking makes one wait. Without adequate isolation you get a lost update.
7. *"Why does an interviewer care whether you'd denormalize?"* — Because it tests whether you measure before optimising, rather than following rules blindly.
8. *"What's the cost of adding an index?"* — Slower writes and extra disk space, since every insert and update must maintain it.
9. *"What does a view actually store?"* — Nothing. It's a saved query that re-runs each time. A materialized view stores results and can go stale.
10. *"Explain a database you've worked with."* — Have one real example ready — even a college project. Say what the tables were, one design decision you made, and one thing you'd change now.

That last one is asked more often than any technical question in this list, and it's the one candidates prepare least.

Recap: Six pairs cover most rapid-fire questions, each separated by a single word — minimal, what-it-depends-on, rollback, when, changed-vs-appeared, and physical.